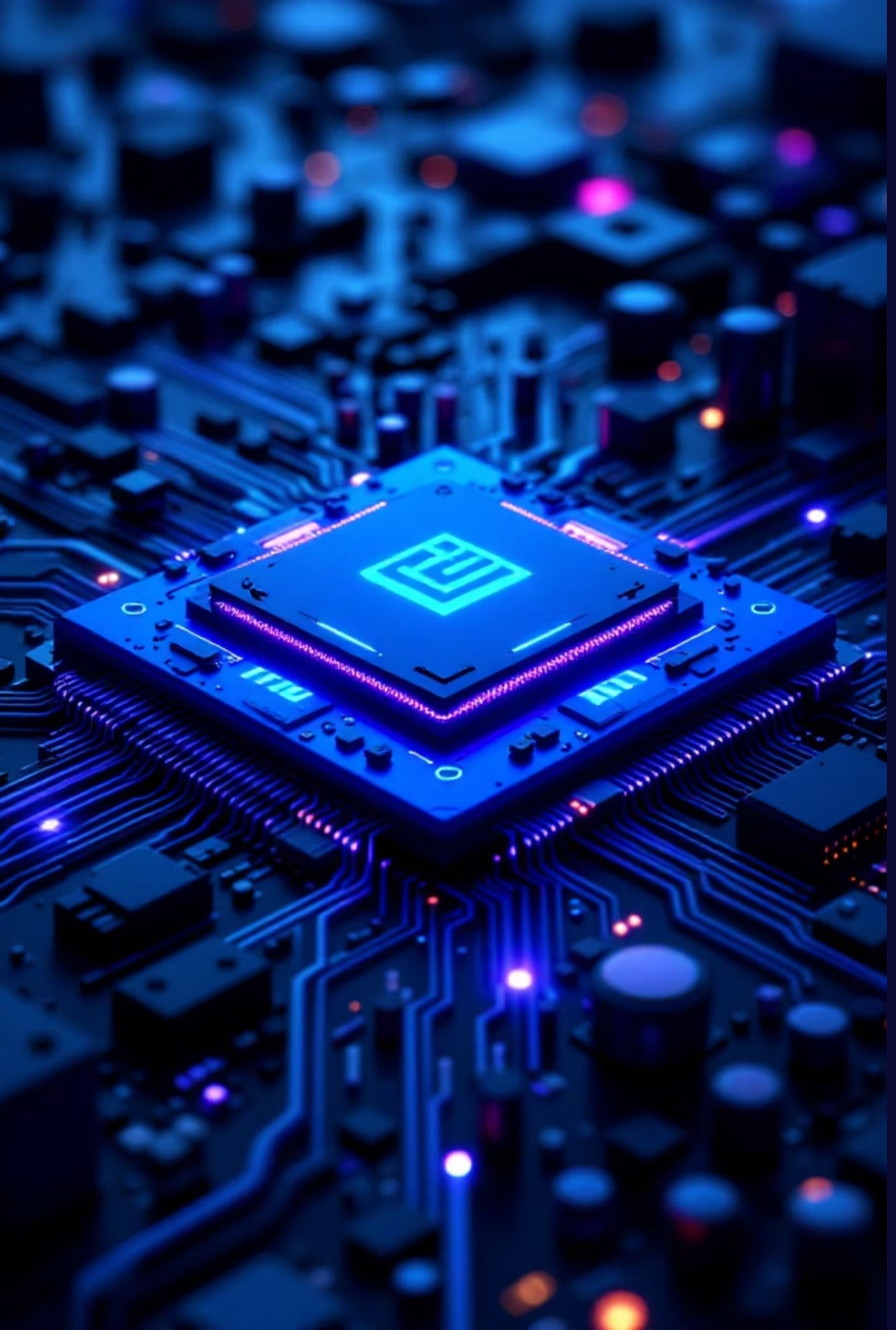


Comparative Analysis of Single Cycle and 5-Stage Pipelined Processor using RISC-V

Presented by:

- Nida Parvin
- Sana Nasrin
- Thesneem V T
- Muhammed Sinan V T

Department: Computer Science and Engineering

Institution: Royal College of Engineering and Technology

Made with GAMMA

Introduction to Processor Architecture

Processor architecture forms the fundamental foundation that determines the overall performance and efficiency of computing systems. The design choices made at the architectural level directly impact execution speed, power consumption, and system throughput.

In this analysis, we examine two distinct processor design methodologies that represent different approaches to instruction execution. Each architecture offers unique trade-offs between complexity, performance, and resource utilisation.

Single Cycle Processor

One instruction per clock cycle with all stages completed sequentially

5-Stage Pipelined Processor

Multiple instructions in parallel through stage-based processing

RISC-V instruction set architecture serves as our implementation platform, enabling direct comparison of architectural differences, execution characteristics, and performance metrics.

Made with GAMMA

Research Objectives

01

Design and Simulation

Develop functional models of both single cycle and pipelined processors using standardised RISC-V instruction set architecture

02

Performance Comparison

Conduct comprehensive analysis of execution time, instruction throughput, and clock cycle requirements for identical instruction sequences

03

Architectural Analysis

Understand fundamental differences in processor organisation, data path management, and control unit complexity

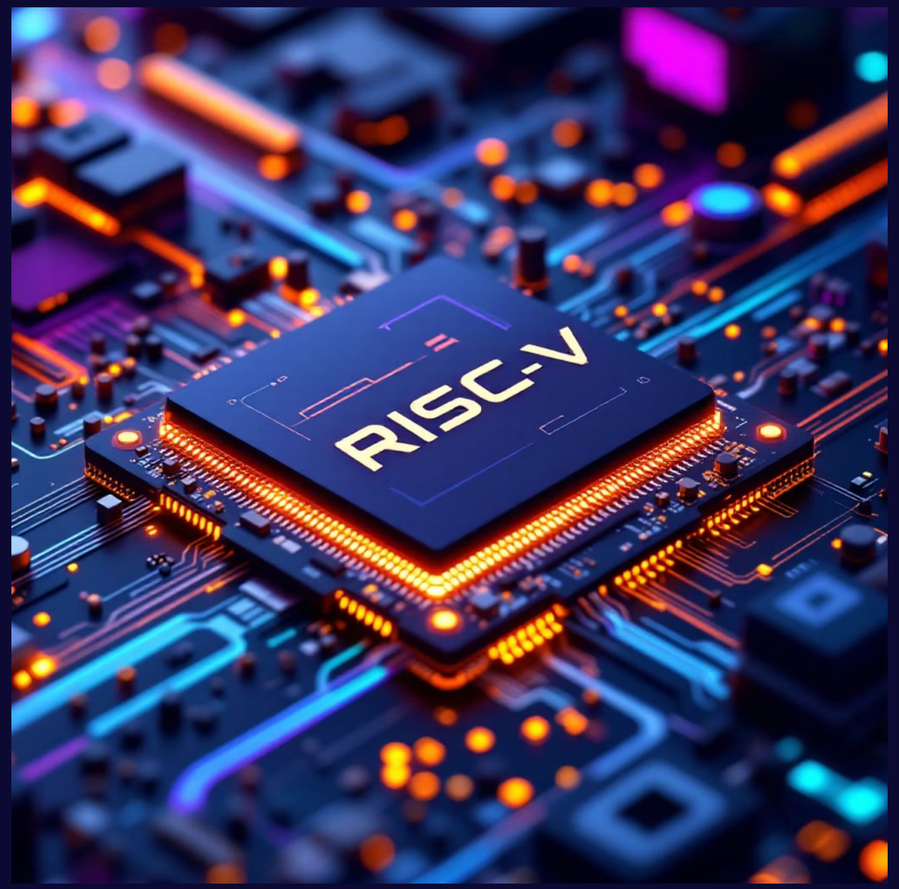
04

RISC-V Implementation

Analyse performance characteristics using real RISC-V assembly instructions and measure practical efficiency gains

Made with GAMMA

About RISC-V Architecture



Open-Source ISA

RISC-V represents an open-source Instruction Set Architecture that has gained widespread adoption in both academic and industrial settings. Its open nature allows unrestricted implementation and modification.

RISC Principles

Built upon Reduced Instruction Set Computing principles, RISC-V emphasises simple, efficient instructions that execute in predictable time frames, enabling optimised pipeline design.

Simple Design

Minimal instruction set with regular encoding format reduces hardware complexity

Modular Structure

Base integer instructions with optional extensions for floating-point, vectors, and atomic operations

Flexible Implementation

Adaptable to various performance requirements from embedded systems to high-performance computing

RISC-V's educational value lies in its transparency and accessibility, making it ideal for teaching processor design concepts and enabling hands-on experimentation with architectural variations.

Made with GAMMA

Single Cycle Processor Architecture

The single cycle processor represents the most straightforward approach to instruction execution, where each complete instruction cycle completes within exactly one clock period. This simplicity comes at the cost of clock speed.

Fetch
Instruction retrieval from memory

Decode
Instruction parsing and operand extraction

Execute
ALU operation and computation

Memory
Data memory access if required

Write Back
Result storage in destination register

Key Characteristic: All five stages execute sequentially within a single clock cycle, requiring the clock period to accommodate the slowest instruction's complete execution time. This results in simpler control logic but longer clock cycles and reduced overall performance.

Made with GAMMA

Single Cycle Processor Operation

Instruction Fetch

Program counter value addresses instruction memory, retrieving the complete instruction word and incrementing PC for next instruction

Instruction Decode

Control unit decodes instruction opcodes, reads register operands from register file, and extracts immediate values

Execute Stage

Arithmetic Logic Unit performs specified operation using decoded operands, computing results and condition flags

Memory Access

Load or store operations access data memory using computed effective address, transferring data between memory and processor

Write Back

Computed result written to destination register in register file, completing the instruction execution entirely within one cycle

The entire instruction processing pipeline completes before the next clock tick, ensuring simple timing but limiting maximum clock frequency to accommodate the slowest instruction path.

Made with GAMMA

5-Stage Pipelined Processor Architecture

Pipelining revolutionises processor design by overlapping instruction execution stages, allowing multiple instructions to progress through different pipeline stages simultaneously. This creates an assembly line effect within the processor.

1

Instruction Fetch (IF)

Retrieve instruction from memory and update program counter

2

Instruction Decode (ID)

Decode instruction and read register operands

3

Execute (EX)

Perform ALU operation on operands

4

Memory Access (MEM)

Access data memory if required by instruction

5

Write Back (WB)

Store result in destination register

Parallel Execution: Each pipeline stage processes a different instruction simultaneously. While instruction N completes write back, instruction N+1 processes memory access, instruction N+2 executes, instruction N+3 decodes, and instruction N+4 fetches. This overlapping significantly improves instruction throughput.

Made with GAMMA

Understanding Pipelining Concept

Assembly Line Analogy

Processor pipelining mirrors automotive assembly line manufacturing, where different workstations process different vehicles simultaneously. Each stage specialises in one operation while multiple products move through the line.
This parallelisation dramatically increases production rate compared to completing one vehicle entirely before starting the next.

Improved Throughput

One instruction completes per clock cycle after pipeline fills, versus one instruction per multiple cycles in single cycle design

Stage Independence

Each pipeline stage operates independently, allowing concurrent processing of multiple instructions at different execution phases

Efficiency Gain

CPU utilisation increases significantly as stalls are minimised and instruction completion rate approaches clock frequency

The pipelined approach requires careful handling of data hazards, control hazards, and structural hazards to maintain correctness while achieving performance benefits. Pipeline registers between stages store intermediate results and control signals.

Made with GAMMA

RISC-V Assembly Code Implementation

The following RISC-V assembly code sequence serves as our benchmark for comparative analysis between single cycle and pipelined processor implementations:

```
addi x1, x0, 5
addi x2, x0, 10
add  x3, x1, x2
addi x4, x0, 3
add  x5, x3, x4
```

Instruction 1

`addi x1, x0, 5` — Load immediate value 5 into register x1

Instruction 2

`addi x2, x0, 10` — Load immediate value 10 into register x2

Instruction 3

`add x3, x1, x2` — Add x1 and x2, store result in x3

Instruction 4

`addi x4, x0, 3` — Load immediate value 3 into register x4

Instruction 5

`add x5, x3, x4` — Add x3 and x4, store result in x5

This sequence includes both immediate loading instructions (`addi`) and register-register arithmetic operations (`add`), providing a representative mix for performance evaluation and hazard analysis.

Made with GAMMA

Code Execution and Final Results

1. **Initial State**
   x0 = 0 (hardwired zero register), all other registers undefined
2. **After Instructions 1-2**
   x1 = 5, x2 = 10 (immediate values loaded)
3. **After Instruction 3**
   x3 = x1 + x2 = 5 + 10 = 15
4. **After Instruction 4**
   x4 = 3 (immediate value loaded)
5. **Final Result**
   x5 = x3 + x4 = 15 + 3 = 18

Register Final Values

- x1 = 5
- x2 = 10
- x3 = 15
- x4 = 3
- **x5 = 18 ✓**

Performance Analysis

The pipelined processor demonstrates superior throughput by overlapping instruction execution stages. After the initial pipeline fill latency, one instruction completes per clock cycle, significantly outperforming the single cycle design.

Key Insight: The final output value x5 = 18 validates correct instruction execution in both architectures. The pipelined processor achieves this result with substantially reduced total execution time through concurrent stage processing.

Made with GAMMA

Performance Comparison Summary

Single Cycle

- Simple design with predictable timing
- Long clock cycles accommodate worst-case instruction
- Lower instruction throughput
- Less efficient CPU resource utilization
- Best suited for educational purposes

Pipelined

- Complex design with multiple stages
- Shorter clock cycles improve timing
- Higher instruction throughput
- Better CPU resource utilization
- Essential for modern processors

The pipelined processor demonstrates superior performance characteristics, particularly for large programs requiring extensive instruction execution. The architectural complexity is justified by substantial performance gains.

Made with GAMMA

Advantages of Pipelining

1

Increased Instruction Throughput

Multiple instructions processed simultaneously across pipeline stages, achieving one instruction completion per clock cycle after pipeline fill

2

Efficient CPU Resource Utilization

Parallel execution ensures processor components remain active, minimizing idle time and maximizing hardware utilization

3

Reduced Execution Time

Shorter clock cycles combined with instruction-level parallelism dramatically reduce total program execution time

4

Superior Modern Processor Performance

Enables complex instruction sets and high-frequency operation essential for contemporary computing requirements

Made with GAMMA

Key Performance Insight

After Pipeline Fill

One Instruction Per Cycle

Pipeline Fill Phase

Initial cycles required to fill all pipeline stages (N cycles for N-stage pipeline)

Steady State Operation

Once filled, processor completes one instruction per clock cycle indefinitely

Maximum Throughput

Achieve theoretical peak performance with continuous instruction completion

This fundamental principle explains why pipelining delivers exceptional performance improvements. The initial pipeline fill overhead becomes negligible for large programs, yielding sustained high throughput.

Made with GAMMA

Architecture Comparison: Conclusion

Single Cycle Processor

Strengths

- Simple and intuitive design
- Easy to implement and debug
- Predictable timing behavior
- Minimal hardware complexity

Weaknesses

- Slower overall performance
- Inefficient resource usage
- Longer clock cycles
- Limited scalability

Pipelined Processor

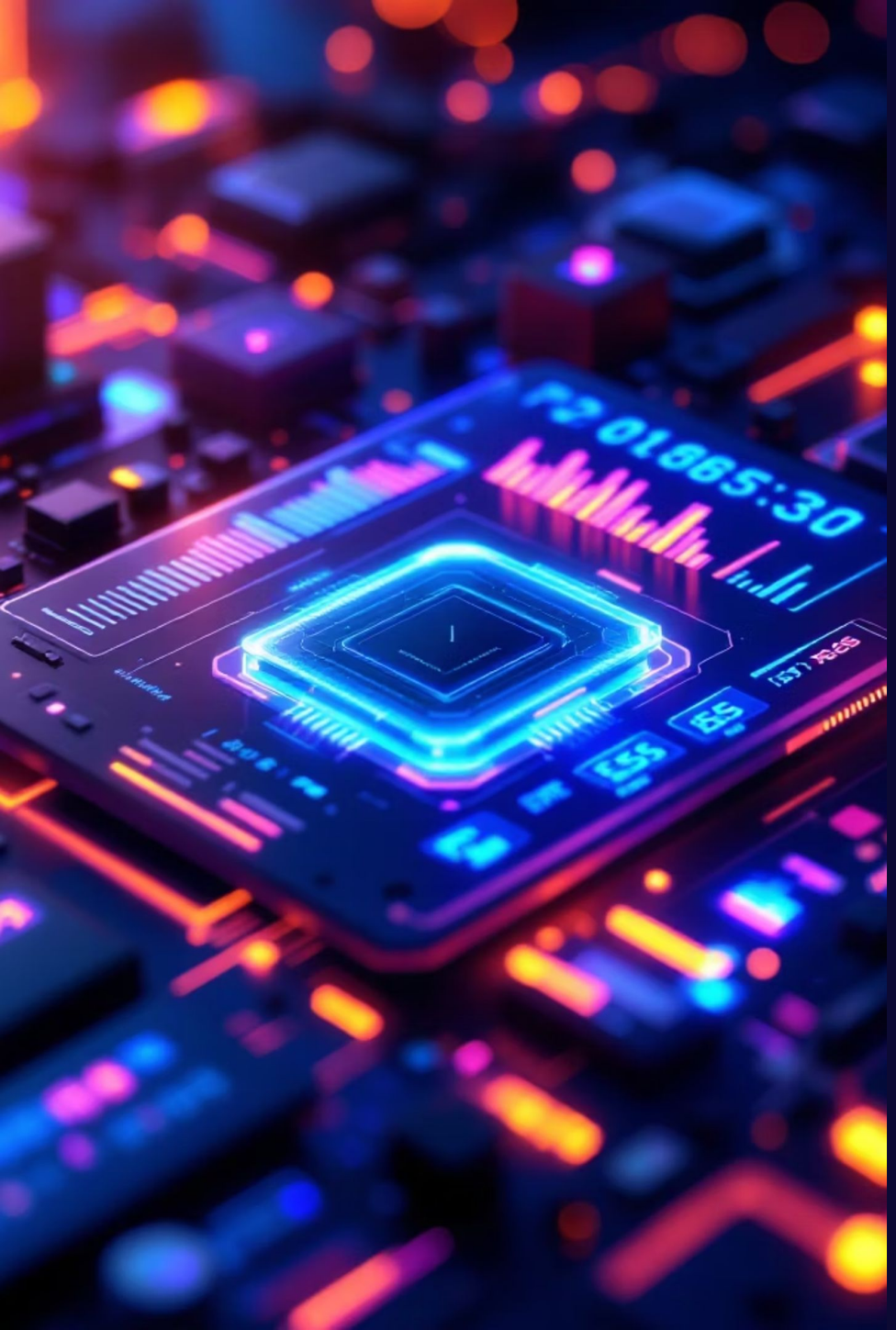
Strengths

- High instruction throughput
- Excellent CPU utilization
- Shorter clock cycles
- Superior performance

Weaknesses

- Complex design and implementation
- Requires hazard handling
- Increased design overhead
- Debugging challenges

Made with GAMMA



Final Observations

Correct Functional Output

Both architectures produce identical results (x5 = 18), demonstrating functional correctness regardless of architectural choice

Performance Disparity

Pipelining delivers substantially faster execution through parallel instruction processing and shorter clock cycles

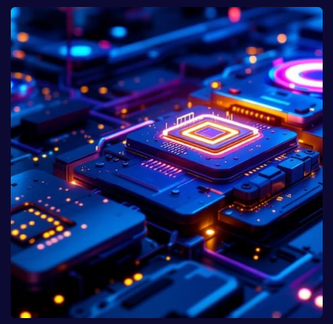
Design Trade-off

The choice between architectures depends on requirements: simplicity versus performance optimization

Practical Insight: Modern processors universally adopt pipelined architectures because performance requirements justify the increased design complexity. Single-cycle designs remain valuable for educational purposes and specialized applications where simplicity outweighs performance needs.

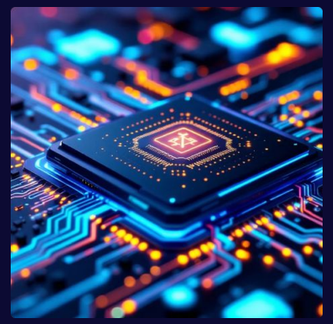
Made with GAMMA

Future Scope and Advanced Topics



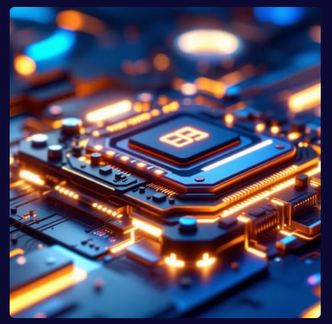
Advanced Pipeline Stages

Implementation of deeper pipelines with 7-stage, 10-stage, or more complex configurations for specialized processors



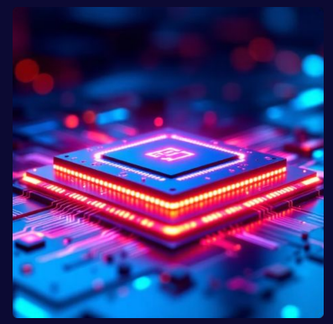
Hazard Handling Techniques

Integration of sophisticated forwarding, stalling, and branch prediction mechanisms to eliminate pipeline stalls



Real-Time System Integration

Application in time-critical systems requiring guaranteed response times and deterministic behavior



Power Optimization

Development of energy-efficient pipelined architectures balancing performance with power consumption constraints

These advanced topics represent the frontier of processor architecture research, enabling next-generation computing platforms for cloud computing, artificial intelligence, and high-performance computing applications.

Made with GAMMA

Thank You

Thank You

We sincerely appreciate your time and attention. This presentation provides foundational knowledge for understanding processor architecture trade-offs and performance optimization principles.

Questions & Discussion

Thank you for your engagement and interest in computer architecture. We welcome your questions and feedback.

Made with GAMMA